

Club Subscriptions

Just a reminder to check the details on your newsletter label.

Please let us know if any of the details are incorrect, and please note the date in the bottom left-hand corner ...

This is the **EXPIRY DATE** of your subscription

It is your subscriptions which keep the club running, therefore if you want the support of the Group to continue, please help us by checking your subscription renewal date.

If your subs are due before the end of the year, please forward them to the Club's P.O. Box 26050 - Newlands, WELLINGTON.

Remember - the subs are still only:

\$18.00 For NZ Residents
NZ\$30 For Australasian Subscriptions
NZ\$40 For other areas overseas.

PUBLIC DOMAIN BASIC - AND RADIO BROADCAST

The difficulties which we have been experiencing with getting the Public Domain Program to air, combined with the limitations of the media:

- A time limitation of 2 Minutes - which isn't much at SV Cassette Basic speed.
- The limited broadcast area (although the broadcasts were picked up from the Wairarapa and other areas well away from the Harbour Capital here -)
- The limited 'audience' (From Access Radio's Point of view)
- The considerable amount of time required to prepare a tape of Transmission Quality (Jim has had to spend the best part of a day preparing material, and trying to ensure that each tape goes to air)
- The fact that no other computer clubs now use the broadcast facilities.

has meant that we will be discontinuing broadcasts. SORRY !!!

BUT REMEMBER . . . ANYTHING THAT WAS BROADCAST CAME FROM THE
P U B L I C D O M A I N

And ALL Public Domain material is available free of charge (except a small handling fee) from the Club P.O. Box - - - >

PUBLIC DOMAIN SOFTWARE

We were very fortunate recently in obtaining a new batch of public domain (BASIC) software, some of which has become public after being 'hidden away' for some time.

Don Stanley (formerly Action Computers) has generously donated the following :

Spectravideo	Super Artist
Spectravideo	Graph Plotter
Spectravideo	Enhance
Spectravideo	Quest Adventure

Some of these have been reviewed in past issues of our newsletter.

However for those who don't know about them, Super Artist converts your computer into a superb full scale artists easel. There is extensive on line help, the ability to dump pictures to cassette disk or printer. The programme itself features many new ideas, such as super fast ram saves and restores, copy part of picture and many more.

Graph Plotter is a 2 dimensional function plotter and numerical analysis system. It is suitable for anyone who needs to plot graphs, either at school or work. The numerical analysis routines include Equation Solving, Integration, Differentiation and Linear System Solving for up to 10 equations.

Enhance is software which produces new Basic commands such as Switch, Kill, Set, Copy and Sprite. This one is not for beginners but will show any MC nuts a thing or two!

Quest is a real good adventure game.

The documentation for Don's programmes is about 13 pages long and is available on a CP/M disc in ASCII form.

Another Don (Rogers by name) has decided to give us his DISCAT programme. This is a programme which will enable you to keep an up to date catalogue of all those discs you have. It runs under Basic and will save you hours of searching.

We have been sent a whole disc of new software from Canada which contains all kinds of goodies. It is mostly high class utilities including the following :

3D-LABYR	is a 'rooms' type puzzle
EDITORDSK	New Disc track, sector editor
3D-TICTAC	O's & X's
VALID DSK	Validates discs (great)
WINDOWSBS	A windows demo (good)
L-FRMTBAS	New logical format.
BACHMUSIK	Music

NSWP BAS	File utility (excellent)
BANK21BAS	Switch utility
PARAJP	Game
BINCASBAS	BSAVE addresses tape.
Q-BERT	Game
CANCEL	?
RENAMEDSK	Goes with Discat
CHECK TXT	Text search prog.
STRIP REM	Strips '
D-KONGBAS	Gorillas!
SUPERS	Game
DISCAT	New Disc cataloguer
SWITCH	MC file

Plus more on this excellent disc.

One more from a local member is MACODE. This is a complete menu driven 'mini' programmers tool. It will read headers from tape or disc (BSAVE addresses), examine and edit memory etc.

The ones which I've run are all good and many new programs hints and tips can be got from these programmes.

I have not yet compiled these programmes onto tapes or discs but I will do my best to accomodate your needs wherever possible. To get all of the above files plus documentation you will need to send me three formatted discs.

Costs are as normal i.e \$3.00 for each disc or tape plus \$2.50 for postage and handling.

Send your requests to

SV P.D. SOFTWARE
P.O Box 26050
NEWLANDS, WELLINGTON
NEW ZEALAND.

Public Domain CP/M

Anyone wanting anything from the library of CP/M PD software, please write to Ross Fletcher at the club address, specifying either which disc, or which programs you would like. The listings in previous issues are printed from double-sided disks therefore as most drives are single sided, a whole disc from the list now occupies up to 2 full discs. Most of the programs on the discs are 'Squeezed', this means they have been run through a function in New Sweep (NSW, Nswep etc. but not the earlier Sweep) so as to take up as little room as possible. They all have a 'Q' as the 2nd letter in their extension, and can be easily unsqueezed by using New Sweep to tag them and then the 'Q' option to unsqueeze them, usually onto another disc. Please send formatted discs, rather than asking us to purchase them for you. Please also include, either a stamped return mailer, or add 60c to your payment to cover postage. We will return your disk in your packaging.

Question and answers column.
From a suggestion by D.Stanley.

In answer to the query about bad sectors or Bad Allocation table, I have recently come up with one solution. Like everything else it is easy once you know how, but this is not a necessarily a fast job or one for absolute beginners.

You will need to have the Basic program called 'SCAN' by L.Dunning or DISKED.BAS and know how to use them. These are both public domain. You also need Don Stanley's program SYSTRACK and maybe the recently added one on the Canadian PD disc called VALID.DSK.

(If you HAD a copy of Brian Fisk's Disc utility and you HAD a back up directory saved on the disc you may not be up this particular stream without a paddle!)

DON'T WRITE ANYTHING TO THE DISC AFTER THE BAD ALLOCATION ERROR. ALWAYS USE THE COMMAND 'CLOSE:FILES' NEVER USE JUST 'FILES'. THIS WILL SOMETIMES SAVE BAD ALLOCATION ERRORS.

1) Use Systrack or maybe Valid.dsk to find out where your bad program files start on the disc. You will also be told where the next tracks are in the case of a large file. If this bombs out then you probably have bad sectors as well. Try VALID.DSK to repair these but if thats no good give up now.

2) Use SCAN to find out which sectors your file occupies. This can take hours and hours. You now know all the tracks and sectors your file occupies.

3) Enter and save the following program but do not run it on the bad disc.

```
10 CLEAR 1000
20 FIELD #0,128 AS A$,128 AS B$
30 OPEN "2:FIXIT1" FOR OUTPUT AS #1
40 FOR SEC=C TO D
50  C$=DSKIS(1,T,SEC)      ' T=TRACK NUMBER
62  LSET C$=A$;RSET C$=B$
74  PRINT #1,A$;B$;
80  NEXT SEC
90 PRINT #1,CHR$(26)
95 CLOSE
```

4) Enter in the appropriate track and sector numbers above. (C,D,T). It's best to do one track at a time, but the old hands can add another FOR/NEXT loop for multiple tracks.

5) Now run the program with your bad disc in No.1 drive and a new disc in No.2. It is possible to use one drive, but writing to your bad disc is not really on. Each time you run this on your bad disc change the suffix number after FIXIT. If you don't then each time you run it the last file created will disappear (same name).

6) Each time you create a file, the directory entry on track 20, sectors 1-3 (disc in drive 2) is written to. The tenth byte of each 16 byte entry is the file attribute (type). In the files you have just created this will be 00H, meaning ASCII. But, I hear you say, "my original file was binary."

7) Now use SCAN again to change this file type byte to 80 Hex. Using SCAN this will now look like an inverse 0. If your original file was ASCII then you don't need to bother with this step. MC files look like an inverse ! ; screen dumps like an inverse '.

8) You should now have a disc with all (or nearly all) of your valuable file on it but in several bits. It only remains now to join them together using either MERGE or APPEND. Use the public domain file called FILE (Disc 17) to APPEND files if your not sure how to do it.

9) Get a copy of Brian Fisk's Disc utility and also change your function key setup or IPL program to make No.1 F/key read "CLOSE:FILES" + CHR\$(13). Use it often.

NOTE : I have not covered ALL the pitfalls to this process. However if you have a 'play' disc handy and a little time you can find out a lot about our primitive (by today's standards) DOS.

As a further follow up to this technique I have written a program to transfer ASCII files directly from CP/M discs to Basic called CP/M COPY which appears in this issue.

----- VIDEO RAM DEMO

When you have a few minutes try this little program :

```
10 L$=INPUT$(1):N=ASC(L$):N=N*8+2303:CLS
20 SCREEN 0
30 FOR L=1 TO 8:A=VPEEK(N+L):A$="":FOR J=0 TO 7:M=A AND
  1:W=32+M*169:A$=CHR$(W)+CHR$(W)+CHR$(W)+A$:A=(A-M)/2:NEXT J
  :PRINT SPC(14)A$:NEXT L:PRINT:PRINT:GOTO 10
```

(Input only lower case 'a' to 'z' until you get the hang of it)

Now that you've tried it perhaps you can explain how it works!
This was adapted from an article in COMPUTE! December '86.

***** A NEW GAMES SCORE

I HAVE BEEN ADVISED THAT THERE IS AN NEW CHAMPION IN THE 'FRANTIC FREDDY' PLAYERS ARENA ...

Alexander Lindsay has topped the old score (21,720) at practise level with a new score of 25,340 < =====

This suggests that maybe I should start a new 'Best Scores' section for the newcomers - please let me know any 'new' scores!

Review on Disk Utility

by Darryl Alison

This disk utility program by Bryan Fisk is a very good utility, designed for a single disk drive. The program is menu driven to allow ease of use and it covers most of those typing errors that we make. The option that are allow by this program are view files on disk, sort disk directory, condense a program, change file protection, load and save back-up directory.

Most of the options speak for themselves ie view files will show all the files on the disk plus their file type and attribute. Sort directory sorts the directory into alphabetical order and then resaves it to the disk, beware if you are using an 80 column card the sort will not appear to work but it does work under 40 columns.

Condense will condense any ascii saved program. It will also give you three options for codensing the program and they are leave remarks alone, leave rem statements only, and to remove remarks and line only if line consists of a remark.

You are also able to change the disks' or a files attribute with ease with the file protection option. You can change your attribute to read only, read / write, or verify. The last option on the program is a very handy one in my books and the is load / save back-up directory. This option loads or saves a back-up directory to the disk so if something dreaded happens (touch wood it doesn't) like a bad allocation table you can load up your back-up directory and cure the error, handy eh!

So in conclusion this disk maagement program is a very good one despite being for one drive. I have also found it very informative in how the computer runs the disk drives just by looking at the program, it is very well laid out for studing thank-you Bryan.

TWO USEFUL DISK-BASIC ROUTINES

Here are two programs from Bryan Fisks' disk utility program that you can include in your program to cover against the dreaded bad allocation table. If you trap error=61 with an on error goto and you use the disk use the first program to recover a corrupted allocation tabe and use the second one to save a back-up directory ever time you use the disk.

LOAD BACK-UP DIRECTORY

```
field#0,255asbu$:fordc=1to3:tps=dski$(drive,20,dc+4):lsetbu$=tp$:
dsko$drive,20,dinc:next:tps=dski$(drive,20,8):fordc=15to17:dsko$
drive,20,dc:next:return
```

SAVE BACK-UP DIRECTORY

```
field#0,255asbu$:fordc=1to3:tps=dski$(drive,20,dc):lsetbu$=tp$:
dsko$drive,20,dc+4:next:tps=dski$(drive,20,15):lsetbu$=tp$:dsko$
drive,20,8
```

Sometimes it is convenient to print out your program listings in double columns. Most basic programs have lots of waste space on the right hand side of the page so this both saves paper and makes it easier to follow the logic if the whole thing is on one page. The program given below is one originally developed by Don Rogers in response to my request for something in basic that would do the same thing as the CP/M PD file called DBL-EP.COM. Don came up with the ideas and I got them working!

The program which you want to print out must be on the same disc and be in ASCII format. (SAVE "1:file",A)

The program is presented here in the form it produces itself when 'working on' any file.

```

10 REM Two column print out of
20 REM files saved in ASCII fo
30 REM D.Rogers./J.A.Oliver 22
40 CLS:REM This is version No.
50 PRINTTAB(5)"PROGRAM MUST BE
60 CLEAR 8000:PL=56:ML=60:C=65
70 LPRINT CHR$(15)
80 PRINT TAB(4)"ENTER NAME OF
90 OPEN "1:"+F$ FOR INPUT AS
100 LN=0
110 LN=LN+1

120 LINE INPUT #1,L$(LN)
140 IF LEN (L$(LN)) <= ML THEN
150 L$(LN+1)=".. "+MID$(L$(LN)
160 L$(LN)=LEFT$(L$(LN),ML)
175 IF LN=PL THEN 140
200 P=INT(LN/2)
230 NEXT I
240 IF E AND (LN<2*P) THEN LPRINT
260 REM Allow for possibility of
.. printed
270 IF LN=2*PL THEN LN=0 ELSE

```

It was produced in this form by writing the strings (lines 220,240 etc) to a disc file instead of to the printer. This ASCII file was then transfered to CP/M. The variables were altered as follows, PL=22, ML=30, C=35. You can of course print to the screen, in case you wish to cram as much as possible onto it. Neat eh ?

***** 80 Column Cards

We have had another request for info about 80 column cards - which have of late been 'scarce as hen's teeth'.

If you happen to have one that you are prepared to sell - please contact:

Dave Browning
73 Kauri Street
TOKOROA

... OR PHONE (0814)64-782 after 6pm

NOTE: I HAVE HEARD THAT BRAND NEW - CUSTOM-MADE 80 COLUMN CARDS ARE NOW AVAILABLE FROM - IMPAK COMPUTING

P.O. BOX 3153
CHRISTCHURCH

BUT I HAVEN'T HEARD OF ANYONE USING ONE. (IF YOU HAVE GOT ONE AND ARE HAPPY WITH IT - LET US KNOW !!!)

IPL Sector on Disk

by Darryl Alison

The IPL function pertains to one sector of the disk i.e. sector 14 of track 20, track 20 is also your directory track. When you boot up disk basic at the end of the loading of disk basic the computer will check the IPL sector and load it into the buffer and execute it. This then enables you to write a little program, up to a maximum of 255 bytes long into that sector.

You can for example get rid of the ipl program to change the function keys and write it into the IPL sector and save a track on the disk. To use the IPL command you must build your program into a string, this cause difficulties like speech marks which are normally used to close a string, this can be over come by this method e.g. A\$="key 2,"+chr\$(34)+" load "+chr\$(34)+"!:". To write the string to the IPL sector you just enter ipl A\$. Some strings using this method will be longer than 255 bytes even though they will not use 255 bytes on the IPL sector, that is because your "+chr\$(34)+" represents 12 bytes in your string but only one byte on the IPL sector, with that in mind here is a sample program that sets the function keys for two drives and runs a named program.

```
10 clear1000
20 A$="width80:key 1,"+chr$(34)+" files "+chr$(34)+"+chr$(13):
   key 2,"+chr$(34)+"load"+chr$(34)+"+chr$(34)+"+chr$(34)+"!:"
   +chr$(34)+"key 3,"+chr$(34)+" save"+chr$(34)+"+chr$(34)+"
   +chr$(34)+"!:"+chr$(34)+"
30 B$=":key 6,"+chr$(34)+" files2"+chr$(34)+"+chr$(13):key 7,
   "+chr$(34)+" load "+chr$(34)+"+chr$(34)+"+chr$(34)+"2:"+chr$(
   34)+"key 8,"+chr$(34)+" save "+chr$(34)+"+chr$(34)+"+chr$(
   34)+"2:"+chr$(34)+"irun"+chr$(34)+"!:"YOUR PROG"
40 ipl A+B$
50 clear200
```

Line 10 clears enough string space so you can work the ipl program line 20 and 30 are your two strings which set up your ipl string. If you don't have an 80 column card just remove the width80 in line 20. Please remember in line 30 to put in at the end the name of the program that you wish to run, if you don't wish to run a program then remove back to the : before the run statement. Line 40 saves your two strings to the IPL sector and line 50 restores the string space.

DBASE PROGRAMMERS HELPER - QUICKCODE

GRAEME DICK has a copy of the QUICKCODE program which is causing him some concern, if you have a copy and can render some assistance, please contact him at the next meeting or through the committee.

CP/M FILE CONVERSION TO BASIC

BY Jim Oliver

Wordstar in 40 columns! Well not quite, but read on if you want a fast and easy way to transfer your CP/M ASCII files to a Basic disc, the screen, printer or tape. WS on tape!!

SV computers use the Xerox 820/2 disc format. The same format is used for Basic and CP/M discs. This may not be readily apparent due to differences in the way CP/M and Basic discs are handled. This means that all discs formatted for SV are recognised by the disc controllers as being alike even though the directories etc are in different tracks.

This means that under Disc Basic we should be able to at least read CP/M discs even if we cannot actually run programs from them.

The trick is in being able to read the CP/M directory information and pass this on and interpret it for use under Basic for transferring files etc.

The CP/M directory track is number 3. Each track (under CP/M) is divided logically into 34 sectors of 128 bytes each. There is a thing called an allocation group in CP/M; this consists of 8 sectors or 1K. All CP/M files are accessed by the system using these 'groups'. Under Basic a 'group' is 4 sectors of 256 bytes each. Simple? Well that's not the whole story but it's all you need to know to understand the following program.

A program such as this is not strictly necessary since you can print or save files under CP/M (not to tape though!). However this was developed in response to a suggestion by a few people in the club as to how it should be done. Well here's one way, and it's faster than using BASIC.IT.

Steve Hanson PROMISED (Well ... 'Hoped' was a better term - Steve =ED) to have the alternative version to this program ie. Basic ----> CP/M ready by next newsletter !!!!!!!

Make sure you have your Basic disc in the 'A' drive and the CP/M disc in the 'B' drive before you run it.

```
10 '  
20 '  
30 ' | CP/M TO BASIC DISC |  
40 '  
50 '  
60 '  
70 ' J.A.Oliver SVUG Wellington New Zealand September 1987
```

```

80 :
90 CLEAR 6000:DIM F$(64),G$(64),L$(64):NF=0
100 :
110 ' Define relationship between CP/M Allocation Groups & BASIC
    sectors
120 :
130 DEF FN S(G)=(G*4) MOD 17 +1
140 DEF FN T(G)=(G*4)\17 + 3
150 FIELD #0, 128 AS A$,128 AS B$ 'ensure ALL 256 bytes
160 ON STOP GOSUB 730:STOP ON
170 CLS
180 :
190 ' Read CP/M directory sectors
200 :
210 GOSUB 590
220 :
230 ' Select output device
240 :
250 CZ=0:WS=0:FLAG=0:CLS:PRINTTAB(5)"Please indicate which device
    you wish to copy to, (D)isc , (P
    )rinter , (S)creen , (T)ape":INPUT DV$
260 DV=INSTR("DdPpSsTt",DV$):ON DV GOSUB 690,690,700,700,710,710,
    720,720
270 IF DV=0 THEN CLS:GOTO 250
280 :
290 ' Display group numbers, file names
300 :
310 CH$="":PRINT:INPUT"Do you want the Allocation Groups display
    ed ";CH$:ON INSTR("NnYy",CH$) GOTO 320,320,340,340:
320 CLS:PRINT"      FILE      NAME      !      FILE NAME      !
    FILE NAME":PRINT STRING$(80,"-"):FOR T=1 TO NF:PRINT
    T," ",F$(T),:NEXT T
330 FOR D=1 TO INT(NF/2):LOCATE 22,D:PRINTCHR$(124):LOCATE30,D:PR
    INTCHR$(124):NEXT D:PRINT:GOTO 360
340 CLS:PRINT"*** FILE      NAME ***":TAB(30)"*** GROUP      ALLOCAT
    IONS ***":TAB(70)"F/Size"
350 FOR T=1 TO NF:PRINT T," ",F$(T)," ":FOR H=1 TO 12:PRINTUS
    ING"****":ASC(MID$(G$(T),H)):NEXT H:PRINT " ",(ASC(L$(T))*12
    B)/1024,"K":NEXT T
360 PRINT:PRINT "Which file do you wish to copy (1-";NF;")":INPUT
    SF:IF SF<1 OR SF>NF THEN 360
370 INPUT "Is this a Wordstar file";WS$:IF INSTR("NnYy",WS$) > 2
    THEN WS=1 ELSE WS=0
380 IF FLAG=1 THEN PRINT: INPUT "What name for this file ";FT$
390 :
400 ' Prepare to write to BASIC disc,Directory track and File All
    ocation Table
410 :
420 OPEN DV$+FT$ FOR OUTPUT AS #1 'prepare BASIC disc
430 FOR AG=1 TO 12 'each allocation group no
    . in turn
440 G=ASC(MID$(G$(SF),AG)):IF G=0 THEN 540'get ascii value, if 0
    then end?
450 SE=FNS(G)-1:TR=FNT(G)

```

```

460   FOR S=1 TO 4                                '4 sectors per allocation
      n group
470   SE=SE+1:IF SE > 17 THEN SE=1 : TR=TR+1      'change tracks
      if s>17
480       D$=DSKI$(2,TR,SE):IF INSTR(D$,CHR$(&H1A))>0 THEN CZ=1
490       LSET D$=A$:RSET D$=B$
500       IF WS=1 THEN GOSUB 780
510       PRINT #1,A$;B$;                          'print ALL 256 bytes
520       IF CZ=1 THEN GOSUB 730                    'end of file ^Z ?
530       NEXT S                                    'next sector
540   NEXT AG                                       'next allocation group
550   GOSUB 730
560   :
570   ' CP/M Directory track read
580   :
590   FOR SEC=1 TO 8:C$=DSKI$(2,3,SEC)
600   FOR I=1 TO 8 :A=ASC(MID$(C$,I*32-31))
610   IF A>90 THEN 670:REM >90 either deleted file or illegal
620   NF=NF+1
630   F$(NF)=MID$(C$,I*32-31,12)                  'file names
640   G$(NF)=MID$(C$,I*32-15,12)                  'group numbers
650   L$(NF)=MID$(C$,I*32-16,1)
660   PR=ASC(MID$(F$(NF),10)):IF PR>128 THEN MID$(F$(NF),10) = CHR$
      (PR-128):REM inverse chars. for file allocation NSWP 2.07
670   NEXT I:NEXT
680   RETURN
690   DV$="1:":FLAG=1:RETURN
700   DV$="LPT:":RETURN
710   SC=1:DV$="CRT:":RETURN
720   DV$="":FLAG=1:RETURN
730   CLOSE:PRINT:PRINT"For another go press any key , to quit pres
      s <esc>":RE$=INPUT$(1):IF RE$<>CHR$(27) THEN RETURN 250
740   PRINT"END":END
750   :
760   'Strip parity bit from Wordstar file bytes
770   :
780   FOR T=1 TO 128:PRINT #1, CHR$((ASC(MID$(A$,T)))AND127);:NEXT
      T:FORT=1TO128: PRINT #1, CHR$((ASC(MID$(B$,T)))AND 127);:NEXT
      T:RETURN 520

```

The program provides most of the prompts necessary which you should answer with a 'y' or 'n'.

The section which converts the WS files to readable material (line 770) is quite slow and gives the reader a chance to scan the text on screen before accepting it for transfer to a disc. Any files over 16K in length will require 'two bites at the cherry' so to speak. These appear in the original display as two distinct files with the same name. They can be APPENDED under Basic if required.

 EDITORS NOTE - To complete the boxed header above, you needed to use the ':' character. - You may be interested to find out that this is obtained by holding down the <LEFT GRAPH> key and pressing the '[' key (Courtesy of "THAT'S FAIRLY INTERESTING" Dept)

SPECTRAVIDEO DISC DIRECTORY UTILITY.

BY JIM OLIVER

If you are like me and spend too much time searching for disc files while other people put you to shame with neatly typed lists of programs, read on.

Since I have around 40 discs and I'm a reluctant typist what I needed was a utility which would read the directory tracks of Basic and CP/M discs, sort them into ASCII order, write the file to disc, allow me to view it, and then if necessary print the file names out.

Below you will see a program to do this.

It is presented here in a semi-compressed form with no REMS. I will give a brief description of the main parts.

Lines 10-80 initialise and present the main menu screen.

90-140 reads Basic disc directory tracks and puts the result into an array. The subroutine starting at line 400 does the same for CP/M discs. Some CP/M file name extension letters are saved to disc as true ASCII plus an offset depending on the exact nature of the file type. You will need to change this to suit your particular system. Most people however will find this program OK.

Once the program is running in memory you can load up both drives and then read the directories very quickly. It took about 12 minutes to read 40 discs. The action slows down when the array and string space fills up, so don't despair when things seem to stop for a while to clean up string space etc.

Line 160 tests for the size of the array and if over 350 entries forces you to save it. This could be changed to suit your conditions.

The subroutine at lines 500 on sorts the list into ASCII order and line 190 saves it to disc then returns to the menu.

The sort routine is a 'Quick sort' type and therefore line 545 will not print out all the file numbers or in any particular order. This line simply informs the user that in fact something is happening. 100 entries takes about 17 seconds. (An earlier version used a bubble sort and took 60 seconds.)

Lines 240-330 print the file out. 270 & 280 contain a statement that works out the correct number of spaces the printer should insert so as to achieve neatly lined up columns. This may need changing to suit the number of letters in your particular disc I.D. Line 290 checks for lines per page and if necessary prints 10 blank lines so as to skip the perforations on line feed paper. Again this may need altering. Line 220 checks the printer port for a 'switched on printer.'

Lines 340-380 display the file on an 80 column screen. The statement in line 360 allows for 3 file names across the screen. Again, depending on your I.D this may need to be changed to achieve neatly lined up columns.

When one screen full has been displayed you are expected to press a key for the next lot.

I have got into a habit of using lower case file names for all Basic discs. This means my print outs have all the CP/M files at the beginning since these are always in caps and are of lower ASCII value than lower case. You may care to separate your file lists.

```

10 REM DISC DIRECTORY READ,SAVE & PRINT
20 REM J.A.OLIVER JUNE87
30 SCREEN,0:COLOR1,10:CLS:MAXFILES=2:CLEAR9000:DIMF$(360),DN$(40)
   T=1:L=0:C=0:IV$=CHR$(27)+"p":NV$=CHR$(27)+"q":CLOSE: ON STOP
   GOSUB 200:STOP ON
40 PRINT"          ..... "
50 PRINT"          ! DISC DIRECTORY ! "
60 PRINT"          ++++++ "
70 LOCATE10,7:PRINT"1: CREATE DISC DIRECTORY":PRINT:PRINTTAB(10);
   "2: PRINT OUT DIRECTORY":PRINT:PRINTTAB(10);"3: VIEW DIRECTORY
   ":PRINT:PRINTTAB(10)"4: QUIT":PRINT:PRINTTAB(10)"Please select
   a number":INPUT S:ON S GOTO 80,220,340,200
80 PRINT"Log in:";IV$+" which drive 1 or 2 "+NV$;" ";D$=INPUT$(1)
   :PRINTD$:D=VAL(D$):PRINT:INPUT "Disc name or no.:";DN$:PRINT:I
   NPUT "(B)asic or (C)P/M Disc";CH$:ON INSTR("BbCc",CH$) GOTO
   90,90,400,400:PRINT"Please use 'B' or 'C' keys":GOTO80
90 FOR SEC = 1TO3: C$=DSK1$(D,20,SEC): FOR I=1TO16: A=ASC(MID$(C$,
   I,16-15))
100 IFA=0THEN130
110 IFA=255THEN140
120 NF=NF+1:F$(NF)=MID$(C$,I*16-15,9)+" "+"on disc "+DN$
130 NEXTI:NEXT
140 IFNF<0THENEND
150 PRINT"Disc files now read":INPUT"Another disc";ND$
160 IFND$="Y"ORND$="y"THENIFNF<350THENBOELSEGOSUB210
170 CLS
180 GOSUB 500
190 OPEN"1:DISC.DIR"FOROUTPUTAS#1:PRINT#1,NF:FORX=1TONF:PRINT#1,F
   $(X):NEXTX:CLOSE#1:PRINT:PRINT:PRINTTAB(10)"DISC DIRECTORY CRE
   ATED":FORW=1TO2000:NEXTW:CLS:GOTO70
200 COLOR15,4:CLS:SCREEN,1:LOCATE15,10:PRINT"Check files":END
210 PRINTIV$+" WARNING save data now "+NV$:RETURN150
220 PRINTTAB(7);"IS THE PRINTER SWITCHED ON!":FOR T=1 TO 80:NEXT
   T:PRINT:IFINP(&H12)AND1 THEN 220:"If byte 0 of port &h12=1 the
   n warn the user to switch on the printer
230 PRINT:PRINTTAB(7)"PRESS ANY KEY FOR PRINTOUT":PR$=INPUT$(1)
240 OPEN"1:DISC.DIR"FORINPUTAS#1:INPUT#1,NF: LPRINT "TOTAL NO. OF
   FILES ARE ";NF
250 T=T+1
260 INPUT#1,F$(T):LPRINTF$(T);:GOSUB320
270 INPUT#1,F$(T+1):LPRINTSPC(28-(LEN(F$(T))));F$(T+1);:GOSUB320
280 INPUT#1,F$(T+2):LPRINTSPC(28-(LEN(F$(T+1))));F$(T+2);:LPRINT:
   L=L+1:GOSUB320
290 IFL>56THENFORLF=1TO10:LPRINT:NEXTLF:L=0
300 IFT<>NFTHEN250

```

```

310 CLOSE:FORW=1TO2000:NEXTW:CLS:GOTO70
320 IFEOF(1)THENCLOSEELSERETURN
330 LPRINT:CLS:RETURN70
340 CLS:OPEN"1:DISC.DIR"FORINPUTAS#1:INPUT#1,NF
350 INPUT#1,F$(T):PRINTF$(T),:F$(T)=" "
360 C=C+1:IFC>63THENP$=INPUT$(1):C=0:GOTO350
370 IF EOF(1)THEN CLOSE ELSE 350
380 PRINTTAB(10)"END OF FILE ; TOTAL NO. OF FILES ARE ";NF:PRINT:
PRINT:PRINT"Press any key":P$=INPUT$(1):CLS:GOTO70
390 REM subroutine for cp/m discs:note pr=protected extension nam
es (nsweep207)
400 FORSEC=1TO8:CS=DSKIS(D,3,SEC)
410 FORI=1TO8 :A=ASC(MID$(CS,I*32-31))
420 IFA=229THEN480:REM 229=E5 Hex. Deleted file
430 NF=NF+1
440 F$(NF)=MID$(CS,I*32-31,12)
450 PR=ASC(MID$(F$(NF),10)):IF PR>128 THEN MID$(F$(NF),10) = CHR$(
PR-128)
460 IFF$(NF)=F$(NF-1)THENN$=NF-1:REM multiple file entries
470 F$(NF)=F$(NF)+ " "+"on disc "+DN$
480 NEXT:I:NEXT
490 GOTO 140
500 REM Quick sort TRS80
510 S(1)=1:S(2)=NF:T=1
520 IF T=0 THEN RETURN
530 T=T-1:I=2*T:L=S(I+1):M=S(I+2):X$=F$(L):J=L:K=M+1
545 LOCATE10,5:PRINT"Sorting file";K
550 K=K-1:IF K=J THEN 610
560 IF X$<=F$(K) THEN 550
570 SWAP F$(J),F$(K)
580 J=J+1:IF K=J THEN 610
590 IF X$>=F$(J) THEN 580
600 SWAP F$(K),F$(J):GOTO550
610 F$(J)=X$:IF M-J<2 THEN 630
620 I=2*T:S(I+1)=J+1:S(I+2)=M:T=T+1
630 IF K-L<2 THEN 520
640 I=2*T:S(I+1)=L:S(I+2)=K-1:T=T+1:GOTO 520

```

HINTS FOR FURTHER DEVELOPMENT:

If you place a system on all your discs then the determination of a Basic, or CP/M disc could be done automatically.

A Basic disc has the word "Microsoft" on track 0, sector 5, starting at byte 120. A CP/M (V2.2) disc has the letters CP/M-80 on track 1, sector 2, starting at byte 10. You could determine the size of files (Basic Only) using this:

```

10 FOR F=1 TO NO. FILES:OPEN "DRIVE:FILENAME"(F) FOR INPUT AS #1
20 FS(F)=LOF(1):FS(F)=FS(F)*.252 (Kbytes long): CLOSE 1:NEXT F

```


The file sizes in Kbytes are stored in the array FS(F) and are accurate to about 250 bytes (1 sector) but for files over 4K or so this is fairly good. This info could be attached to the file name. Another virtually undocumented SV feature!

The test for no. of files (line 160) could be altered to test for string space remaining i.e FRE(AS). You can then jump to the SAVE routine when the string space remaining is say < 300. This forces the computer to do a 'garbage collection' of the string handling functions and thus the action would slow down at this point.

If you really wanted to go mad you could add code to determine the file type and status, then print this out too.

Newsletter Contributions

**HERE IS YOUR OPPORTUNITY
TO GET INTO ----> PRINT**

There are a lot of you who are writing programs in BASIC that other people would be interested in, so PLEASE send in a brief outline of your program together with your program (on cassette or disk and we can merge it into the newsletter) care of the club PO Box. Any of the following items would be gladly received:

- Programs - Basic, Pascal, Assembly, You Name It.
- Programming hints, and techniques.
- Utilities - any type, as long as they are self explanatory or come with instructions.
- News - anything you think other club members might like to know.
- Problems - we will find someone to answer them if we can't.

Everything we receive will be considered. If you are sending in printed copy make sure that

1. It is printed on A4 paper with 6 lines at the top and bottom and AT LEAST 1 INCH on either side.
2. you use a ribbon that gives DARK print or use emphasised mode

otherwise it may not be possible to print your article. This is because all articles are photo-reduced, and by the time they are printed, they have been copied at least twice - each time the quality is reduced slightly. If the article is too light, it literally disappears.

**WE PREFER ARTICLES
ON DISK OR TAPE
AND PROMISE TO RETURN YOUR MEDIA ... !!!**

CRTERN
A SCREEN BLANKING ROUTINE FOR 40 COLUMN SCREEN
A MACHINE-CODE PROGRAM RUNNING UNDER BASIC

By Bob Curwood of Christchurch

This program was written with high hopes, which were quickly dashed!! The initial concept was quite simple, a machine code routine which is located in high-memory and monitors the RST 38H interrupt from the graphics chip. The idea is to use this to downcount a 5 minute timer at the same time as the keyboard is monitored for any keypress. When the timer reaches zero, which it can only do if no key has been pressed for 5 minutes, the screen is blanked until a key IS pressed.

Several problems arose after I'd thought about it for a while, first, it wouldn't work on the 80 col: screen since that is memory mapped and although it's easy to blank the screen and cursor in machine code, both the contents and the cursor location would have to be saved in memory (perhaps in compressed form) and reloaded when a key is pressed. This really needs a separate routine to itself so if anyone is interest let me know and I'll do it. (A CP/M version is quite feasible too)

The second problem is that the routine to change the colour CHGCLR only works for the text screen.

The third problem is that the routine which gets the keyboard status, CHGNS, returns with the Z flag set and this will not be cleared until the buffer is read, thus if the application program running is waiting for an input and a key has been pressed but enter has not, the timer is being reset to MAX all the time.

I couldn't just clear the buffer each time the RST 38 occurs because you would then be unable to enter anything other than a single key (INKEY\$ would still work OK!)

This was overcome by polling all 89 keys from within the routine and not using the inbuilt ROM routine at all.

The final result does work but only for the text screen. Whether a program is running or not, the screen will blank out to COLOURS 0 on 0 after about 5 minutes and will be restored when any key is pressed. The test program alters the time to 30 seconds to enable the input instructions to be tested.

The time allowed is computed (by the assembler) in the EQUATE MAX, this can be altered to suit, but if you are not using an assembler but working from the HEX dump, the value to alter is the two bytes underlined. Work out what value you need at 50 'TICKS' per second, convert this to HEX and use this value in place of those two bytes (remember to put the 'low-byte' first).

Other routines I'm working on are the equivalent of VPEEK and VPoke for the 80 col: screen, might be useful to someone? ...

Bob.

```

10 *****
20 *** **
30 *** CRTBRN **
40 *** **
50 *** by **
60 *** Bob Curwood **
70 *** **
80 *** Technical Officer **
90 *** of the **
100 *** Christchurch Spectravideo **
110 *** Users Group **
120 *** **
130 *****
140 ORG OD254H
150 LOAD OD254H
160
170 FORCLR: EGU OFA0AH
180 CHGCLR: EGU 03750H
190 CLRBUF: EGU 0351FH
200 HKEYI: EGU 0FE79H
210 TICK: EGU 50
220 MINUTE: EGU TICK * 60
230 MAX: EGU MINUTE * 5
240 ALTRNK: LD DE,START
250 LD HL,HKEYI+2
260 LD A,D
270 LD (HL),A
280 LD A,E
290 DEC HL
300 LD (HL),A
310 LD A,0C3H
320 DEC HL
330 LD (HL),A
340 START: LD HL,(FORCLR)
350 LD A,H
360 OR L
370 JR NZ,NOTBLNK
380 LD A,(FLAG)
390 JR NZ,NOTBLNK
400 IArrive here if screen IS blanked
410 CALL CHKKB
420 RET Z
430 LD HL,(COLOUR)
440 LD (FORCLR),HL
450 CALL CHGCLR
460 LD A,OFFH
470 LD (FLAG),A
480 JR RESTCNT
490 IArrive here if screen is NOT blanked
500 NOTBLNK: LD HL,(COUNT)
510 DEC HL
520 LD (COUNT),HL
530 LD A,H
540 OR L
550 JR NZ,NZERO

;Alter the hook
; called during
; the RST 30H
; interrupt

;Get screen colours
;Check if transparent
; on transparent

;Perhaps it's SUPPOSED
; to be blank?

;Scan keyboard
;Waiting for a keypress
;Get 'old' colours
; and put them back
; with this
;Anything except zero
; indicates not blanked
;Exit via 'normal'

;Is count zero yet?
;No

```

```

560 ;Arrive here if screen is to BE blanked
570 BLANK: LD HL,(FORCLR) ;Get present colours
580 LD (COLOUR),HL ;Save them
590 XOR A ;Transparent 'colour'
600 LD (FORCLR),A ; in foreground
610 LD (FORCLR+1),A ; & background
620 LD (FLAG),A ;Indicates screen blanked
630 CALL CHGCLR ;Blank screen out
640 CALL CLRBUF ;Empty kybd buffer
650 RET ; & wait for a keypress

660 ;Arrive here if timer still ticking
670 NZERO: CALL CHKKB ;During 'normal' use
680 RET Z ; set count to MAX each
690 REStCNT: LD HL,MAX ; time a key is pressed
700 LD (COUNT),HL
710 RET
720 CHKKB: XOR A ;1st row of kybd
730 LD C,A ;Save it
740 KBDLP: OUT (96H),A ;Set it up
750 IN A,(97H) ;Read that row
760 CP OFFH ;FF if no key pressed
770 RET NZ ;Z reset if key pressed
780 INC C ;Next row
790 LD A,C
800 CP 13 ;12 is last row
810 JR NZ,KBDLP ;Next row
820 RET ;Z set if NO keys pressed
830
840 COUNT: DW MAX
850 COLOUR: DW 00 ;Store 'OLD' colours here
860 FLAG: DB OFFH ;Zero = timeout/blanked
870 BINKBASIC: EBU 5
880
890 ;Use DEFUSER-4MD254 for the initial set-up and to start the timer
900 ;Or, after assembly and saving as a m/c file use BLOAD"xxxxxxx",r)
910
920 ;Use POKE 4HFE79,201 to stop timer & POKE 4HFE79,195 to restart
930
940 D254 11 45 D2 21 78 FE 7A 77 78 2B 77 3E C3 2B 77 2A
950 B244 0A FA 7C B3 20 19 3A C4 D2 20 14 CB B2 B2 C8 2A
960 D276 C4 D2 22 0A FA CB 50 37 3E FF 32 C4 D2 18 24 2A
970 B284 C2 D2 2B 22 C2 D2 7C B3 20 17 2A 0A FA 22 C4 D2
980 B294 AF 32 0A FA 32 0B FA 32 C4 D2 CB 50 37 CB 1F 35
990 B2A4 C9 CB B2 D2 CB 21 78 3A 22 C2 D2 C9 AF 4F D3 94
1000 D2B4 DB 99 FE FF C0 0C 79 FE 0B 20 F3 C9 98 3A 00 00
1010 D2C4 FF
1020 END

```

```

10 ' This is the test program for CRTBRN which must be assembled at D256H
20 '
30 ' Bob Curwood Christchurch
40 '
50 ONSTOP80SUB150:ONINTERVAL=180SUB170:DEFUSRI=LHD256' Set it up
60 POKE&HD2AC,220:POKE&HD2AD,5' Make it 30 secs
70 STOPON:INTERVALON' while testing
80 A=USRI(0)' Start it going
90 PRINT'Check that pressing ANY key resets the count to 1500 (30 secs)
100 'AS=INKEY$:IFAS<>' THENPRINT'inkey$="AS' \
110 'AS=INPUT$(1):PRINT'input$(1)="AS' >Test the inputs
120 'INPUT"Type something":AS:PRINT'input="AS' /
130 'LINEINPUT"And again":AS:PRINT'lineinput="AS' /
140 GOTO 100' Do it again
150 POKE&HFE79,201' Turn it off
160 END
170 INTERVALOFF'
180 KEY3,STR$(256#PEEK(&HD2C3)+PEEK(&HD2C2))' >Whats happening?
190 INTERVALON'
200 RETURN' ~ This wont work for the three INPUTs or if STOP is pressed
210 ' but the count IS going down, just wait for 30 seconds and see!
220 '
230 '
240 ' IF
250 ' screen blank
260 ' AND
270 ' flag set (0)
280 ' THEN (Screen has been blanked)
290 ' Scan keyboard
300 ' IF
310 ' key pressed
320 ' THEN
330 ' restore screen
340 ' & clear kybd buffer
350 ' & reset flag (FF) (Back to normal)
360 ' & reset count
370 ' & return
380 ' ELSE (Wait for keypress)
390 ' return (Screen has NOT been blanked)
400 'ELSE
410 ' decrement count
420 ' IF
430 ' count = zero
440 ' THEN
450 ' set flag
460 ' & save screen colour
470 ' & blank screen (Time out, blank screen)
480 ' & clear kybd buffer
490 ' & return
500 ' ELSE
510 ' scan keyboard
520 ' IF
530 ' key pressed
540 ' THEN (Still being used)
550 ' reset count
560 ' & return
570 ' ELSE
580 ' return (You have 5 minutes)

```

ANSWERS TO REMAINING QUESTIONS IN LAST ISSUE

1. ESCAPING FROM GAMES

Few commercial programmers want you to "ESCAPE" from a program without resetting (or powering off) your computer. They fear that "users" (Th...Th... That's US folks !!) will steal routines and ideas from their programs, or make numerous copies. Such (dare I say it ...) Piracy (I'm not entering into the argument that a User should be able to make a back-up of any program he's paid good money for .. really!) is easily possible if you can return to BASIC and then list the program. The easiest way to protect themselves is to:

- use machine code to hide the program so that it cannot be (easily) listed
- Ensure the computer is powered off at the end of the game thus ensuring the program is no longer in memory.

To further complicate matters, programmers mix'n'match all the available protection methods for extra protection.

Only (devious?) machine-code programmers can easily find their way around the protection systems. Unless you can match them reset button if you have one) when you finish your game.

2. SEQUENTIAL FILES AND (EOF(n)) FUNCTION

The best way to explain how to use the (EOF(n)) function is by example:

```

10 ' PRINTS OUT SINGLE CHARACTERS FROM A SERIAL FILE
20 OPEN '1:FILE.TST' FOR INPUT AS #1
30 ' INPUT LOOP STARTS HERE (=====)
40 IF EOF(1) THEN GOTO 100 ' is it EOF?
50 AS = INPUT$(1, #1) ' if not, input some data
60 PRINT AS ' now do something with it
70 GOTO 30 ' go back to repeat test)=-
100 ' PROGRAM CONTINUES IF NO DATA LEFT
110 CLOSE #1
120 PRINT 'FILE END REACHED'
130 END
    
```

NOTE that the (EOF(n)) must be invoked BEFORE any input is attempted. For STRING Input, any of the three Input command formats may be used, depending upon the way the file was originally written (ie (INPUT #n, AS) (LINEINPUT #n, AS) (AS=INPUT\$(1, #n)) where n is the file number and 1 is the length of input string)

3. RANDOM FILES - FINDING THE END OF FILE

The (EOF(n)) function described above is purely for sequential files. To find out the highest RANDOM record written (you should have a variable which keeps track of the records in your database anyway) you can use the (LOF(n)) function:

```

3110 OPEN '1:FILE2.TST' AS #2
3120 R = LOF(2) ' 2 = buffer number used in OPEN above
3130 CLOSE #2
3140 PRINT 'LAST RECORD WRITTEN WAS - ' ; R
    
```

NOTE that the (LOF(n)) function returns the numeric value of the LAST record ~~written~~ written in a RANDOM file. (If used on a Sequential file, this returns a value of the next sector which may be written (and is used for APPENDING))

4. RANDOM FILES - FIELD/RECORD LENGTHS

A RANDOM file consists of a variable number of RECORDS, which are in turn divided into a number of FIELDS using the (FIELD#n,...) statement). The maximum length for a RECORD is 255 bytes. It therefore follows that FIELDS cannot exceed 255 bytes (in fact as each FIELD is represented by a string variable the maximum length is actually 255 bytes). It also follows that where a RECORD is divided into more than one FIELD, the total of ALL FIELDS may not exceed 255. The following program sets up a dummy RANDOM file, then reads it back.

```

2000 CLEAR 1000:MAXFILES=4
2010 OPEN*1:FILE3.TST* AS #3
2020 FIELD #3, 20 AS C$, 40 AS D$, 15 AS E$, 180 AS F$
2030 G$=STRING$(20,"G"):H$=STRING$(40,"H"):
      J$=STRING$(180,"J")
2040 ' A SHORT LOOP TO WRITE A FILE
2050 FOR R = 1 TO 8
2060 R$ = CHR$(&H30+R): I$=STRING$(16,R$)
2070 RSET C$ = G$ ' Note that you do NOT use an
2080 RSET D$ = H$ ' ordinary "X$=Y$" or "LET X$=Y$"
2090 RSET E$ = I$ ' statement as this upsets the RANDOM
2100 RSET F$ = J$ ' BUFFER organisation (LSET is O.K.!)
2110 PUT #3, R ' This writes RECORD "R" to disc
2120 NEXT
2130 CLOSE #3
2200 ' END OF FILE WRITING SECTION

3110 OPEN*1:FILE3.TST* AS #4
3120 FIELD #4, 20 AS K$, 40 AS L$, 16 AS M$, 180 AS N$
3130 FOR R = 1 TO (LOF(4))
3140 GET #4,R
3150 PRINT K$;L$;M$;N$
3160 NEXT
3170 CLOSE #4
3180 PRINT"FILE ENDED AT RECORD ";(R-1)
3190 END

```

NOTE that I have used different file BUFFER numbers (see the OPEN statements) to show that you don't have to use the same file BUFFER when reading a file as you did when writing it. (you can - but you don't HAVE to).

MEMBERSHIP APPLICATION

IF YOU KNOW ANYONE INTERESTED IN JOINING OUR CLUB, PLEASE GIVE THEM A COPY OF THIS APPLICATION ...

FULL NAME _____

ADDRESS _____

Telephones _____ NZ Postal Code _____ (if known)

MEMBERSHIP FEES (If paid during the year July 87 - June 88):

\$18.00 For NZ Members

NZ\$30.00 for Australia & NZ\$40.00 for elsewhere.

For information - Phone ROSS (788-454) or STEVE (843-495)

Post to SV Users Group, P.O.Box 26050, NEWLANDS, WELLINGTON.